



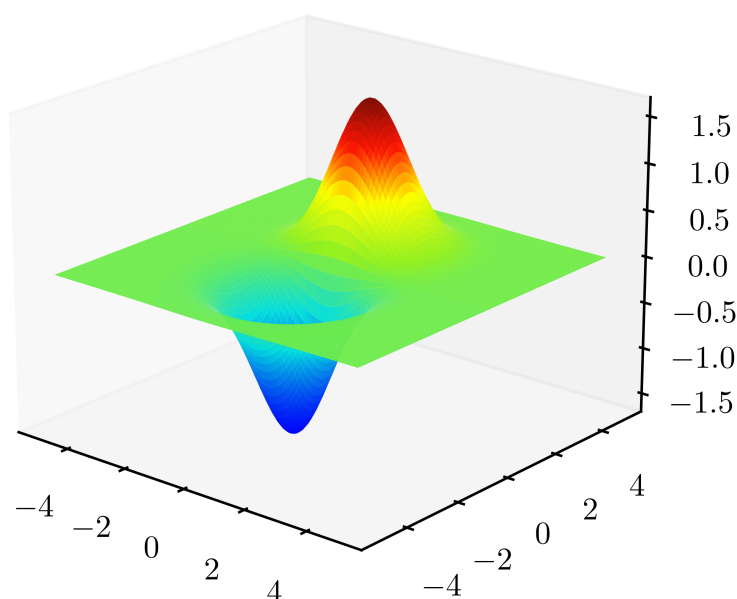
ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
Σχολή Ναυπηγών Μηχανολόγων Μηχανικών

ΥΠΟΛΟΓΙΣΤΙΚΗ ΥΔΡΟΔΥΝΑΜΙΚΗ

Ανάπτυξη Υπολογιστικού Κώδικα για την Προσομοίωση Σύζευξης Στροβίλων Taylor

Ομάδα 76

Αικατερίνη Μαργαρίτα Ηλέκτρα Γεωργιάδη 08118303
Ιωάννης Σκουνάκης - Κουνάβης 08118076



Υπολογιστική Υδροδυναμική - Εργασία 1η

Αικατερίνη-Μαργαρίτα-Ηλέκτρα Γεωργούλη & Ιωάννης Σκουνάκης-Κουνάβης

Εθνικό Μετσόβιο Πολυτεχνείο

Απρίλιος, 2022

Εισαγωγή - Περιγραφή Προβλήματος

Στα πλαίσια του μαθήματος "Υπολογιστική Υδροδυναμική", ο σπουδαστής καλείται να μελετήσει φαινόμενα ρευστομηχανικής με εργαλείο την αριθμητική διακριτοποίηση και την υλοποίηση ανάλογου κώδικα. Η παρούσα εργασία έχει σκοπό την μελέτη της σύζευξης δύο αντίθετων, ίσης έντασης στροβίλων Taylor μέσω της ροικής συνάρτησης ψ και του πεδίου στροβιλότητας ω . Οι στροβίλοι βρίσκονται στις θέσεις $y_1 = 1.5, y_2 = -1.5$ και $x_{1,2} = 0$. Η προσέγγιση της επίλυσης γίνεται μέσω χρήσης πεπερασμένων διαφορών, ενώ απαιτείται η εξέταση και της χρονικής εξέλιξης της σύζευξης.

Μαθηματική Μοντελοποίηση Το μη μόνιμο πρόβλημα περιγράφεται από τις εξισώσεις Navier - Stokes, για διδιάστατη, ασυμπίεστη, συνεκτική ροή.

$$\begin{aligned}\nabla^2 \vec{u} &= 0 \\ \frac{\partial \vec{u}}{\partial t} + (\vec{u} \cdot \nabla) \vec{u} + \frac{\nabla p}{\rho} &= \nu_\alpha \nabla^2 \vec{u}\end{aligned}$$

Με την εισαγωγή της στροβιλότητας ω και της ροικής συνάρτησης ψ οι εξισώσεις λαμβάνουν την ακόλουθη μορφή.

$$\nabla^2 \psi = -\omega \quad (1)$$

$$\frac{\partial \omega}{\partial t} + (\vec{u} \cdot \nabla) \omega = \nu_\alpha \nabla^2 \omega \quad (2)$$

Οι οριακές συνθήκες προκύπτουν μέσω της συνάρτησης Green $G(x, x_0)$. Σημειώνεται ότι στην πραγματικότητα είναι αναλυτική λύση μιας διδιάστατης εξίσωσης Poisson.

$$\psi_B(\vec{r}) = - \int_V G(\vec{r}, \vec{r}_0) \omega(x_0) dV \quad (3)$$

Όπου,

- $G(\vec{r}, \vec{r}_0) = \frac{1}{2\pi} \ln(|\vec{r} - \vec{r}_0|)$
- $\vec{r} \in \partial V, \vec{r}_0 \in V$

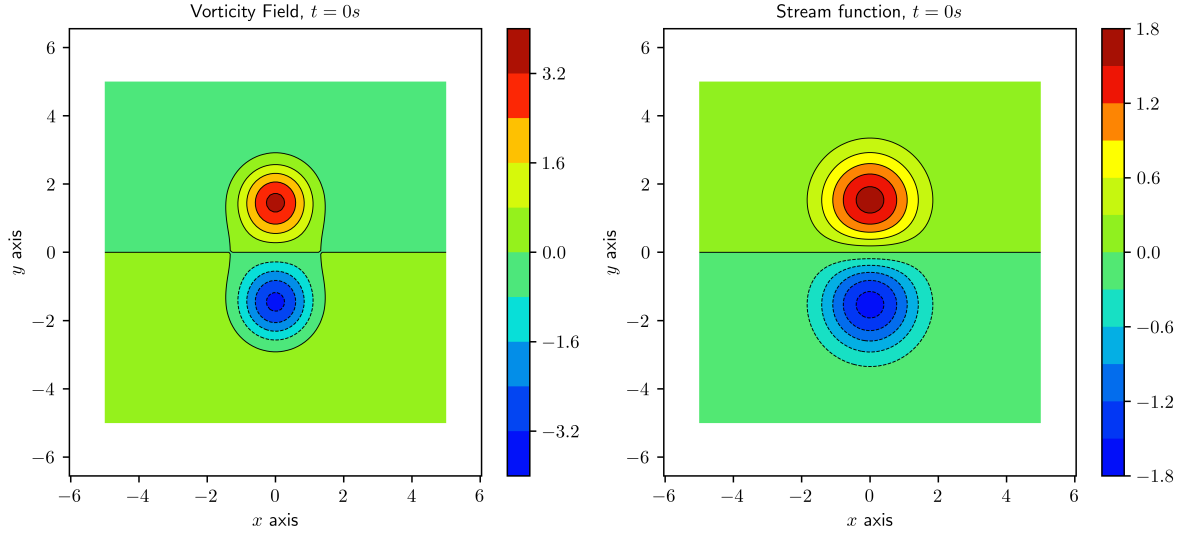
Υπενθυμίζονται επίσης οι εξής σχέσεις:

- $u = \frac{\partial \psi}{\partial y}, v = -\frac{\partial \psi}{\partial x}$
- $\omega = \frac{\partial v}{\partial x} - \frac{\partial u}{\partial y}$
- $|\vec{r} - \vec{r}_0| = \sqrt{(x - x_0)^2 + (y - y_0)^2}$, δηλαδή η απόσταση οριακού κόμβου στη θέση $\vec{r} = (x, y) \in \partial V$ από ένα εσωτερικό σημείο $\vec{r}_0 = (x_0, y_0) \in V$.

Η απόσταση $|\vec{r} - \vec{r}_0|$ μπορεί να προσεγγιστεί και ως η ευκλείδεια νόρμα του διανύσματος $\vec{r}_B = \vec{r} - \vec{r}_0$, δηλαδή $\|\vec{r}_B\|_2$. Ευκλείδεια νόρμα για n-διάστατο διάνυσμα $\vec{x}$:

$$\|\vec{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$$

Analytical Vorticity & Stream Function



Σχήμα 1: Αριστερά το πεδίο στροβιλότητας, δεξιά η ροϊκή συνάρτηση

Για να εξασφαλιστεί η καλή λειτουργία του προγράμματος επίλυσης, δίνονται επίσης οι αναλυτικές εκφράσεις του πεδίου στροβιλότητας (ω_0) καθώς και της ροϊκής συνάρτησης (ψ_0) για $t = 0$ για έναν σπρόβιλο Taylor.

$$\omega_T = (2 - r^2)e^{(1-r^2)/2} \quad (4)$$

$$\psi_T = e^{(1-r^2)/2} \quad (5)$$

Αριθμητική Διακριτοποίηση Για την επίλυση του προβλήματος επιλέγεται ένα χωρίο $S = [a, b] \times [c, d] = \{(x, y) \in \mathbb{R}^2 | a \leq x \leq b, c \leq y \leq d\}$ στον διδιάστατο χώρο, με διαμέριση dx και dy στους αντίστοιχους άξονες. Έτσι παράγεται το πλέγμα με αριθμό στοιχείων N_x στον άξονα x και N_y στον άξονα y τα οποία επιλέγονται αυθαίρετα έτσι ώστε να συγκλίνει η λύση. Παρατηρείται ότι η αρχική κατάσταση δεν έχει κάποια "πυκνότερη" πληροφορία σε έναν από τους δύο άξονες, είναι δηλαδή αρκετά συμμετρική, επομένως επιλέγεται $N_x = N_y$. $dx = \frac{b-a}{N_x-1}$, $dy = \frac{d-c}{N_y-1}$. Στο σχήμα 2 φαίνεται ακριβώς ο τρόπος διακριτοποίησης και αρίθμησης του πεδίου στο οποίο θα υπολογιστεί η λύση.

Παρατηρείται εύκολα, ότι συνυπάρχουν 3 τρόποι αρίθμησης των σημείων του πλέγματος. Ο βασικότερος, και αναγκαίος για την επίλυση, είναι οι ήδη υπάρχουσες καρτεσιανές συντεταγμένες, με την αρχή των αξόνων στο κέντρο του πλέγματος (λόγω συμμετρίας του προβλήματος). Κάθε επόμενο σημείο απέχει Δx ή Δy αντίστοιχα από το προηγούμενο (εδώ, $\Delta x = \Delta y$). Για τον σκοπό της διακριτοποίησης, δημιουργείται ένα νέο σύστημα συντεταγμένων (i, j) με το σημείο $(i=1, j=1)$ στην κάτω αριστερή γωνία του πλέγματος. Στο σχήμα 2 παρουσιάζεται η δομή και των δύο συστημάτων συντεταγμένων. Τέλος, επιλέγεται μια ολική αρίθμηση για την πινακοποίηση των εξισώσεων, τέτοια ώστε:

$$(i, j) = (j - 1)N_x + i$$

η οποία επίσης φαίνεται μερικώς με κόκκινα γράμματα.

Μετά την διακριτοποίηση του πεδίου, διακριτοποιούνται και οι δοσμένες εξισώσεις με χρήση κεντρικού σχήματος για τους εσωτερικούς κόμβους, και προσθήκη των οριακών συνθηκών στους υπολοίπους.

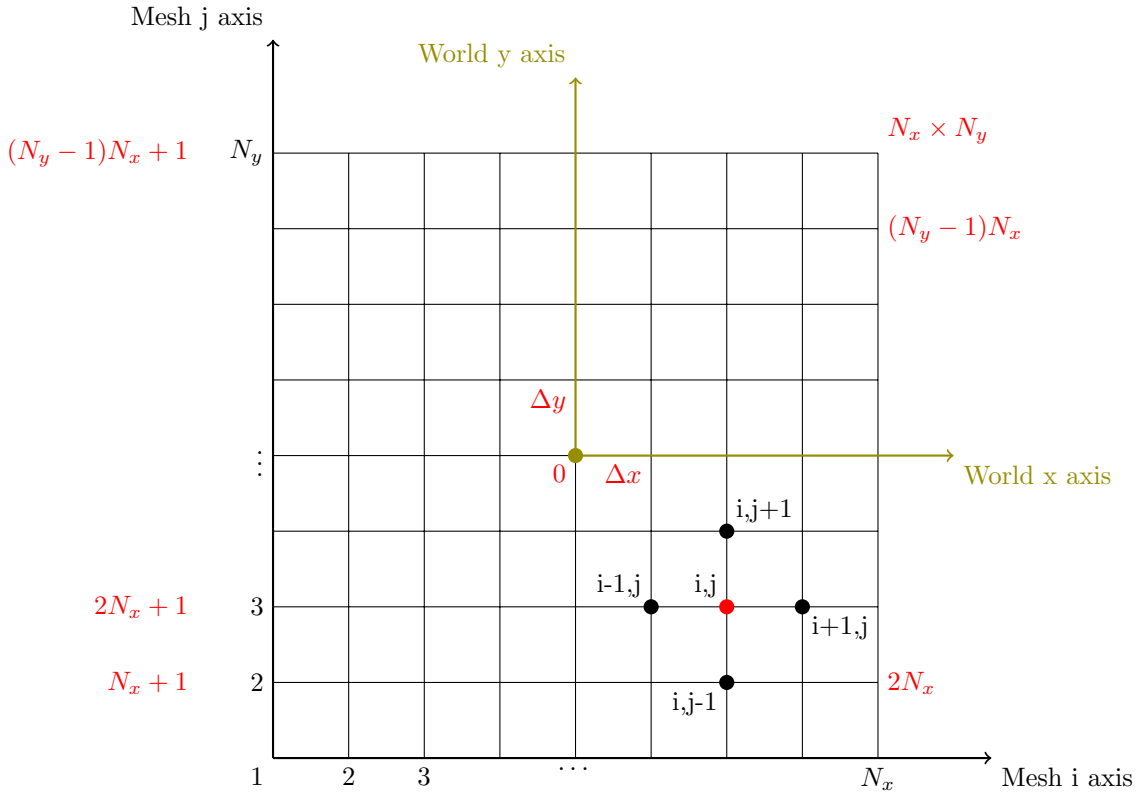
- Εσωτερικοί κόμβοι

$$\frac{\psi_{i+1,j} - 2\psi_{i,j} + \psi_{i-1,j}}{dx^2} + \frac{\psi_{i,j+1} - 2\psi_{i,j} + \psi_{i,j-1}}{dy^2} = -\omega_{i,j}$$

- Οριακοί κόμβοι

$$\psi_B(\vec{r}) = - \sum_{i=2}^{N_x-1} \sum_{j=2}^{N_y-1} \frac{1}{2\pi} \ln(|\vec{r} - \vec{r}_{i,j}|) \omega_{i,j} dx dy$$

Δημιουργείται έτσι ένα γραμμικό σύστημα $N_x N_y \times N_x N_y$ εξισώσεων και αγνώστων της μορφής $[\mathbf{A}]\vec{x} = \vec{b}$, το οποίο περιέχει όλες τις εξισώσεις του πλέγματος. Με την επίλυση του, επιλύεται αριθμητικά η εξίσωση **Poisson**



Σχήμα 2: Διακριτοποίηση του πεδίου επίλυσης

και μπορεί να συγκριθεί με την αναλυτική της μορφή. Σε κάπως πιο ανηγμένη μορφή, φαίνεται το γραμμικό σχήμα παρακάτω. Είναι παραμετρικό καθώς επιτρέπει την επιλογή και ασύμμετρων πλεγμάτων ($N_x \neq N_y$).

$$\begin{bmatrix} 1 & & & & & & & \\ & \ddots & & & & & & \\ & & 1 & & & & & \\ \frac{1}{dy^2} & \cdots & \frac{1}{dx^2} & -2\left(\frac{1}{dx^2} + \frac{1}{dy^2}\right) & \frac{1}{dx^2} & \cdots & \frac{1}{dy^2} & \\ & & \frac{1}{dy^2} & \cdots & \frac{1}{dx^2} & -2\left(\frac{1}{dx^2} + \frac{1}{dy^2}\right) & \frac{1}{dx^2} & \cdots & \frac{1}{dy^2} \\ & & & & 1 & & & \ddots & \\ & & & & & & & & 1 \end{bmatrix} \begin{bmatrix} \psi_1 \\ \vdots \\ \psi_{N_x+1} \\ \psi_{N_x+2} \\ \vdots \\ \psi_{(N_y-1)N_x-1} \\ \psi_{(N_y-1)N_x} \\ \vdots \\ \psi_{N_y N_x} \end{bmatrix} =$$

Στο βήμα της χρονοεπέλασης, δηλαδή ο αριθμητικός υπολογισμός του μη μόνιμου προβλήματος, η εξίσωση (2) διακριτοποιείται με βάση ρητό σχήμα πρώτης τάξης (Euler).

$$\omega_{i,j}^{n+1} = \omega_{i,j}^n - dt \left(u_{i,j}^n \frac{\partial \omega}{\partial x} \Big|_{i,j} + v_{i,j}^n \frac{\partial \omega}{\partial y} \Big|_{i,j} \right) + \nu_\alpha \left(\frac{\partial^2 \omega}{\partial x^2} \Big|_{i,j} + \frac{\partial^2 \omega}{\partial y^2} \Big|_{i,j} \right)$$

- $\frac{\partial \omega}{\partial x} \Big|_{i,j} = \frac{\omega_{i+1,j} - \omega_{i-1,j}}{2dx}$
- $\frac{\partial \omega}{\partial y} \Big|_{i,j} = \frac{\omega_{i,j+1} - \omega_{i,j-1}}{2dy}$
- $\frac{\partial^2 \omega}{\partial x^2} \Big|_{i,j} = \frac{\omega_{i+1,j} - 2\omega_{i,j} + \omega_{i-1,j}}{dx^2}$

$$\bullet \left. \frac{\partial^2 \omega}{\partial y^2} \right|_{i,j} = \frac{\omega_{i,j+1} - 2\omega_{i,j} + \omega_{i,j-1}}{dy^2}$$

Όπου οι εκθέτες αναφέρονται σε χρονική στιγμή, ενώ οι βάσεις σε χωρικό κόμβο. Σημειώνεται εδώ, ότι ενώ οι τιμές των οριακών ψ καθορίζονται από τις συνοριακές συνθήκες, δεν ισχύει το ίδιο και για τα ω . Επομένως η παραπάνω κεντρική προσέγγιση δεν μπορεί να εφαρμοστεί, προτείνεται όμως η εύρεση της πρώτης παραγώγου αριθμητικά με εμπρός ή πίσω διαφόριση, και ύστερα με παραγωγή της αντίστοιχα βρίσκεται η δεύτερη.

Με μια επαναληπτική διαδικασία, και για ένα αρχούντως μικρό χρονικό και χωρικό βήμα, η λύση συγκλίνει και τελικά περιγράφεται πλήρως το μη μόνιμο πρόβλημα.

Υπολογιστικός Κώδικας

Για την ανάπτυξη κώδικα επίλυσης των γραμμικών συστημάτων, και απεικόνισης των αποτελεσμάτων, χρησιμοποιήθηκε η γλώσσα Python και συγκεκριμένα οι βιβλιοθήκες `numpy`, `pandas` και `matplotlib`. Προς επεξήγηση του κώδικα, θα αναλυθούν οι διάφορες συναρτήσεις που δημιουργήθηκαν για να αντιμετωπίσουν διαφορετικά κομμάτια του προβλήματος, ενώ στο τέλος παρατίθενται και τα αποτελέσματα.

Όσον αφορά την γενικότερη ροή του προγράμματος:

- Πρώτα επιλέγονται τα όρια του πλέγματος, τα N_x, N_y, dt, dy , αρχικοποιείται το πλέγμα, και ύστερα υπολογίζονται οι αναλυτικές τιμές ω_0 και ψ_0 .
- Αρχικοποιείται ο πίνακας $[A]$ ως $N_x N_y \times N_x N_y$ με μοναδιαία διαγώνιο και τοποθετούνται οι συντελεστές¹ για τους εσωτερικούς κόμβους.
- Υπολογίζεται ο πίνακας b , τοποθετούνται οι συνοριακές συνθήκες.
- Λύνεται το γραμμικό σύστημα μέσω της `np.linalg.solve()`, και υπολογίζεται το ψ_0 αριθμητικά.
- Με γνωστό πεδίο στροβιλότητας και ροϊκή συνάρτηση, χρησιμοποιείται βρόχος `for` για n χρονικές στιγμές βήματος dt , ο οποίος καλεί την συνάρτηση `updateMESH()`.
- Εντός της συνάρτησης υπολογίζονται τα $u, v, \omega_x, \omega_y, \omega_{xx}, \omega_{yy}$, και εφαρμόζεται το ρητό σχήμα Euler προς εύρεση του νέου πεδίου στροβιλότητας.

Συναρτήσεις

`distance(x1,y1,x2,y2)`

Η ευκλείδεια νόρμα του διανύσματος $(x_1 - x_2, y_1 - y_2)$. Χρήση της `np.linalg.norm()`.

`showgrid(x,y)`

Εμφανίζει στο ενεργό διάγραμμα τις γραμμές του πλέγματος $N_x \times N_y$.

`W0(x,y)`

Αναλυτική έκφραση του πεδίου στροβιλότητας για $t = 0$: $\omega = \omega_1(r_1) + \omega_2(r_2)$.

`psy0(x,y)`

Αναλυτική έκφραση της ροϊκής συνάρτησης για $t = 0$.

`num2coord(n,flag=False)`

Με όρισμα αριθμό ολικής αρίθμησης επιστρέφει τις τιμές i, j , και για `flag = True` επιστρέφει τις πραγματικές συντεταγμένες x, y . Με όρισμα τιμές $[i, j]$ επιστρέφει τις συντεταγμένες.

`deriv(ii,jj,dgrid)`

Προτεινόμενη συνάρτηση παραγωγής, με χρήση `try ... except` statements, δοκιμάζει πρώτα κεντρική, ύστερα εμπρός και τέλος πίσω παραγωγή στον κόμβο ii, jj για συνάρτηση `dgrid` τύπου `pandas.DataFrame`. Αντί αυτής επιλέχθηκε απλά η συνάρτηση `np.gradient(f,dy,dx)` για λόγους ταχύτητας και καθαρότητας στον κώδικα.

Σημειώνεται ότι για την $\frac{\partial^2 f}{\partial x^2}$ μπορούν να βρεθούν όλες οι παράγωγοι σε κάθε κόμβο, και ύστερα να ξαναγίνει παραγωγή τους, αποφεύγεται έτσι το κεντρικό σχήμα της διακριτοποιημένης δεύτερης παραγώγου.

¹ Αν και υποθέτουμε ότι $N_x = N_y$, ο κώδικας χειρίζεται και μη συμμετρικά πλέγματα, για αυτό και ο πίνακας $[A]$ δεν έχει συντελεστές $[1...1 - 4 \ 1...1]$ αλλά αυτούς της εξίσωσης Poisson, $\nabla^2 \psi = -\omega$

ar2mesh(ar,A=False)

Μετατροπή του `ar` σε `pandas.DataFrame` object.

buildb(W)

Αρχικοποίηση του πίνακα `b`, για κάθε $i \in [2, N_x - 1], j \in [2, N_y - 1]$ (εσωτερικοί κόμβοι) τοποθετείται στην αντίστοιχη θέση ολικής αρίθμησης η τιμή $-\omega_{i,j}$.

boundmat(b,w,A)

Προσθήκη των συνοριακών τιμών ψ_B , όπου ο πίνακας `[A]` δεν αναφέρεται σε εσωτερικό κόμβο, (άρα συνοριακό σημείο) υπολογίζεται το διπλό άθροισμα για την συνοριακή συνθήκη και τοποθετείται στην αντίστοιχη θέση στον πίνακα `b`. Αποτελεί το πιο χρονοβόρο κομμάτι της διαδικασίας, αποφεύχθηκε η χρήση `for` και επιλέχθηκαν οι πινακικές πράξεις και τέλος το άθροισμα όλων των στοιχείων. Η διαφορά σε χρόνο εκτέλεσης ήταν της τάξης του 10^2 .

animateVORT(frames)

Απαραίτητη συνάρτηση για χρήση του πακέτου `matplotlib.animation`. Με χρήση της `updateMESH()` δημιουργεί ένα νέο διάγραμμα για κάθε χρονική στιγμή και το χρησιμοποιεί σαν `frame` για την αναπαραγωγή σε βίντεο.

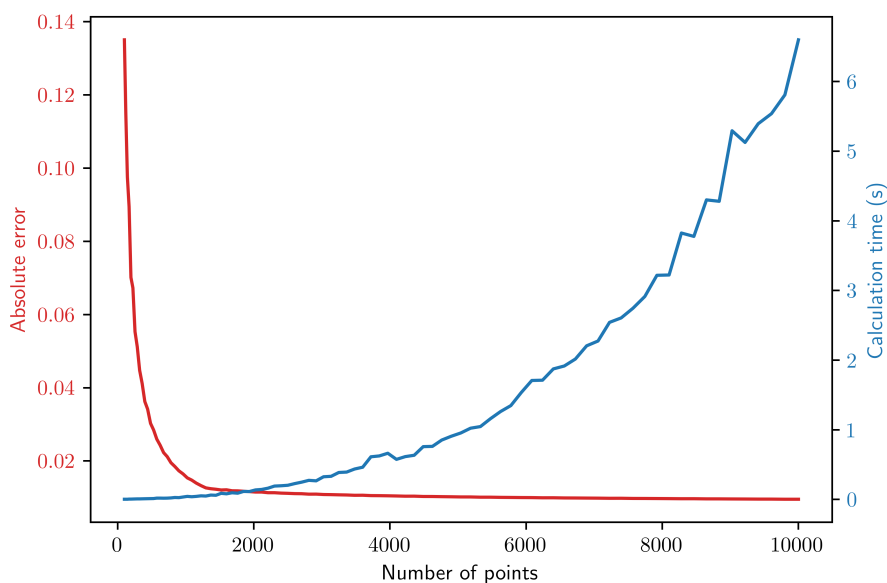
updateMESH(w,psy,plotting=True,viscosity=False)

Βασική συνάρτηση που για κάθε πεδίο στροβιλότητας και ροϊκή συνάρτηση υπολογίζει τα πεδία ταχυτήτων u, v , και τις παραγώγους του πεδίου στροβιλότητας, εφαρμόζει το σχήμα Euler, και επιστρέφει το πεδίο ω της επόμενης χρονικής στιγμής καθώς και την ροϊκή συνάρτηση που προκύπτει από αυτό με επίλυση του νέου γραμμικού συστήματος. Επιτρέπει και συνεχόμενη παραγωγή διαγραμμάτων για κάθε χρονική στιγμή.

Αποτελέσματα & Σχολιασμός

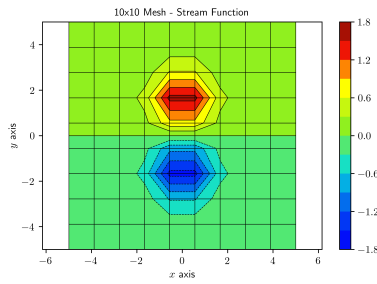
Επίλυση εξίσωσης Poisson

Για την επίλυση του προβλήματος εξετάστηκε γενικότερα η συμπεριφορά της λύσης συναρτήσει του πλέγματος, και μεταφέρθηκε σε γράφημα το μέγιστο απόλυτο σφάλμα, $\max \left(\frac{\psi_{\text{numerical}} - \psi_{\text{analytical}}}{\max(\psi_{\text{numerical}})} \right)$, μαζί με τον χρόνο εκτέλεσης μίας επανάληψης (εύρεση οριακών συνθηκών και λύση γραμμικού συστήματος). Αυτό, μαζί με μερικά παραδείγματα πλεγμάτων και των αντίστοιχων λύσεων που δίνουν, προσέφεραν μια ποιοτική ένδειξη για τα χαρακτηριστικά της λύσης.

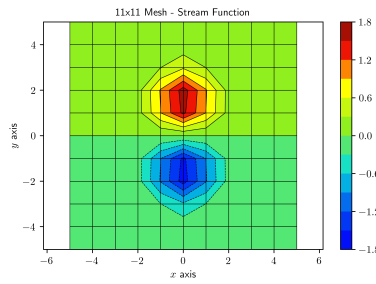


Σχήμα 3: Γράφημα μέγιστου σφάλματος - χρόνου εκτέλεσης συναρτήσει σημείων πλέγματος.

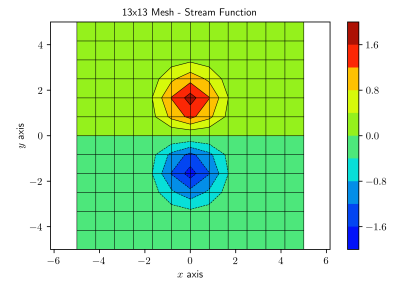
Παρατηρείται ότι το σφάλμα μειώνεται πολύ απότομα ήδη από τα 2000 σημεία ($N_x = N_y \approx 45$), ενώ ο χρόνος εκτέλεσης μόνο της επίλυσης του συστήματος και όχι της χρονοεπέλασης, αυξάνεται ραγδαία. Όσον αφορά το



(a) 10x10 mesh

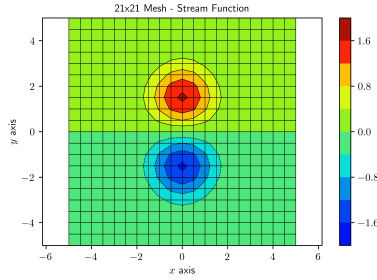


(b) 11x11 mesh

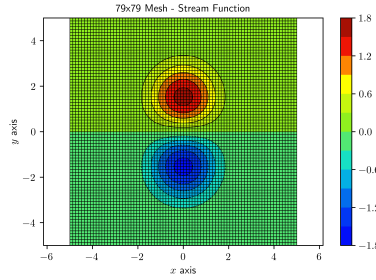


(c) 13x13 mesh

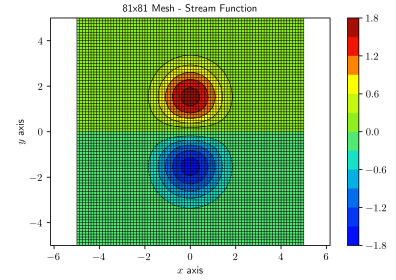
Σχήμα 4: Σχετικά αραιά πλέγματα, με εμφανή επιρροή των σημείων στην μορφή του διαγράμματος



(a) 21x21 mesh



(b) 79x79 mesh



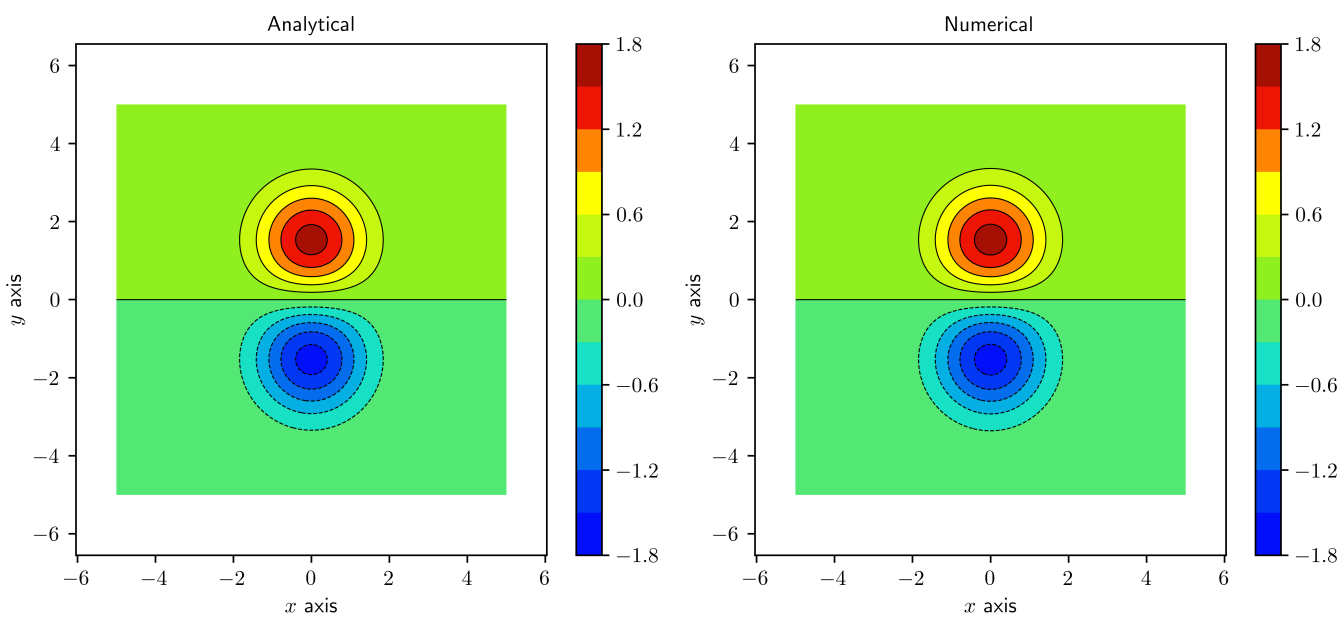
(c) 80x80 mesh

Σχήμα 5: Πλέγματα με σημαντικά μεγαλύτερο αριθμό σημείων

πλέγμα, στο σχήμα 4 παρατηρείται ότι έχει σημασία το αν τα σημεία είναι μονά ή ζυγά λόγω συμμετρίας του προβλήματος (τα κέντρα βρίσκονται στη μέση του εύρους τιμών), αλλά αυτή η συμπεριφορά σιγά σιγά "σβήνει" με περισσότερα σημεία, όπως φαίνεται στο σχήμα 5.

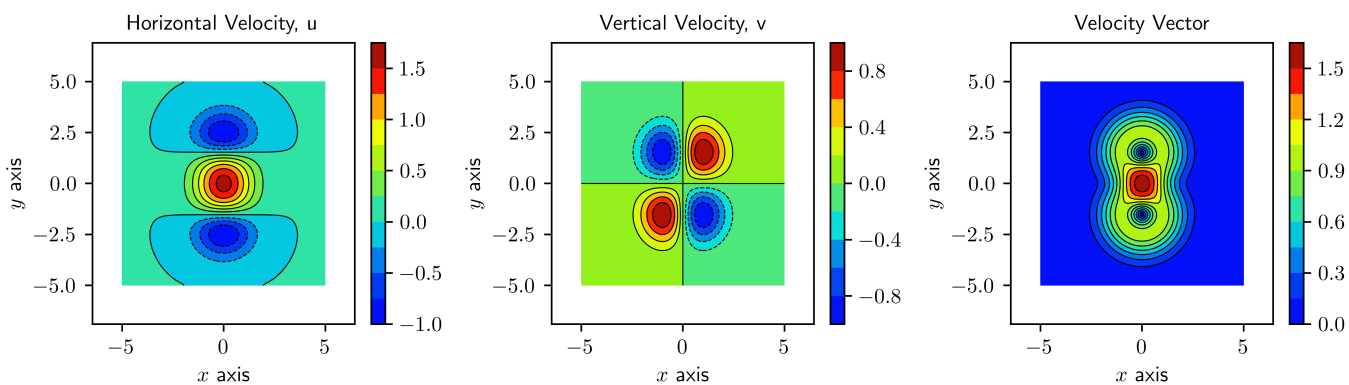
Με βάση τα παραπάνω, για την επίλυση του προβλήματος Poisson επιλέχθηκε πλέγμα 100x100 για μεγαλύτερη ακρίβεια και εύρος $x, y = [-5, 5]$, ενώ για την χρονοεπέλαση επιλέχθηκε 80x80 καθώς η διαφορά στο σφάλμα είναι ελάχιστη, ενώ ο χρόνος εκτέλεσης είναι τάξεις συντομότερος. Τα αποτελέσματα φαίνονται στα διαγράμματα που ακολουθούν. Το σχήμα 6 συγκρίνει το αριθμητικό αποτέλεσμα με την γνωστή αναλυτική έκφραση οπτικά, το σχήμα 8 ποσοτικοποιεί το σφάλμα με ισούψεις γραμμές, και τέλος στο σχήμα 7 παρατίθενται τα προκύπτοντα πεδία ταχυτήτων.

Analytical vs Numerical Stream Function

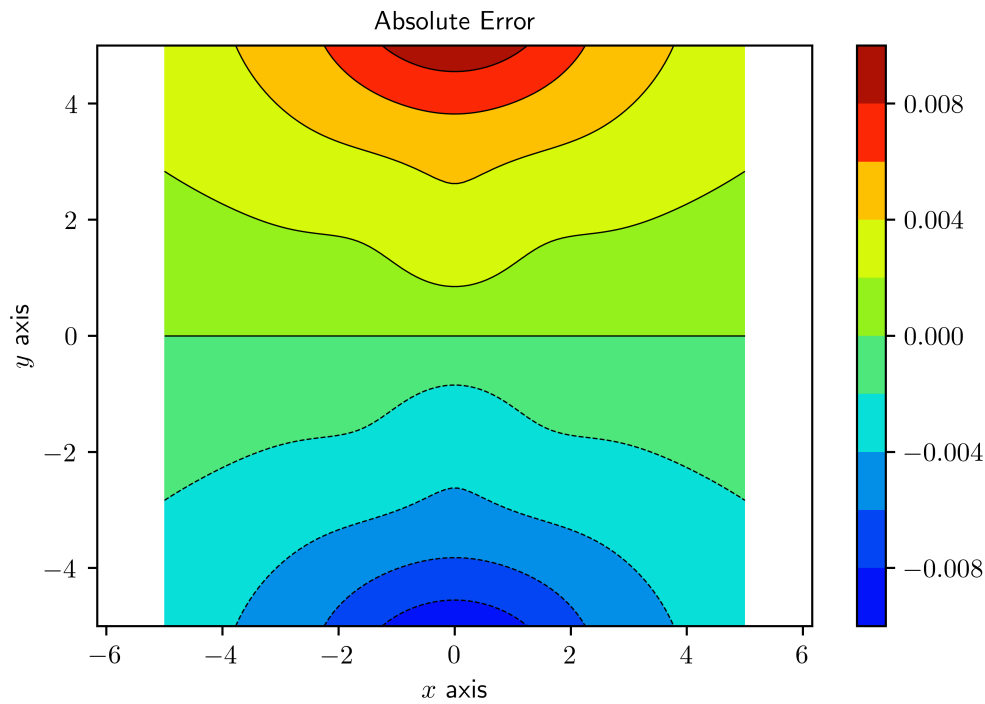


Σχήμα 6: Σύγκριση ροϊκής συνάρτησης ψ , αναλυτικό και αριθμητικό αποτέλεσμα

Horizontal & Vertical Velocity Fields



Σχήμα 7: Πεδία ταχυτήτων για $t = 0s$



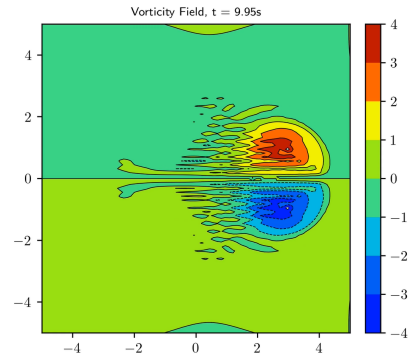
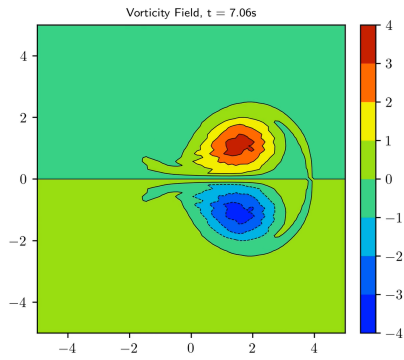
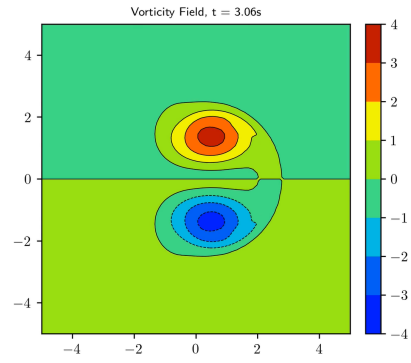
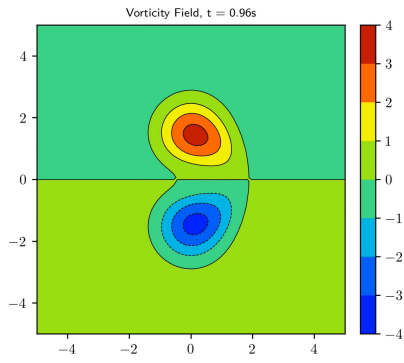
Σχήμα 8: Γραμμές στάθμης σφάλματος

Χρονοεπέλαση - Μη Μόνιμο Πρόβλημα

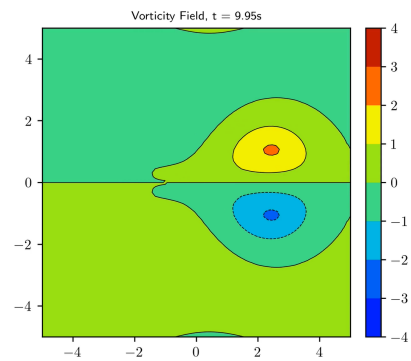
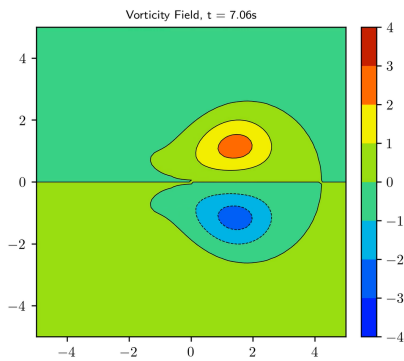
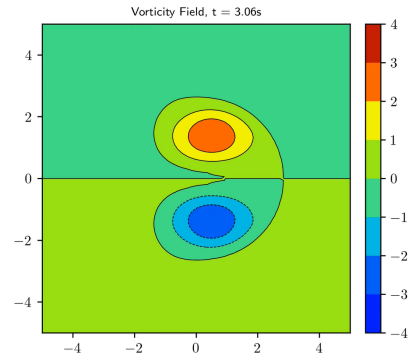
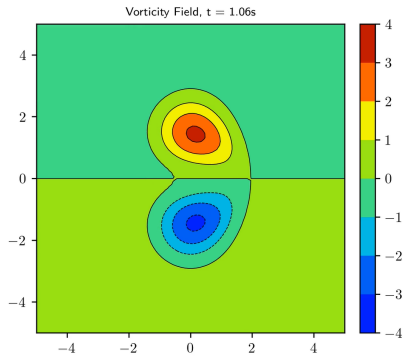
Στην επίλυση του προβλήματος στο χρόνο, εμφανίζονται δύο περιπτώσεις: Το πρόβλημα με ή χωρίς τάσεις τριβής. Στην παρούσα επιλέχθηκε κινηματική συνεκτικότητα $\nu_\alpha = 1.5e - 4$, παρότι προτείνεται η $\nu_\alpha = 1.5e - 5$ του αέρα. Αυτό οφείλεται στο ότι η τελευταία είναι πολύ μικρή έτσι ώστε να εμφανίσει δραστικές αλλαγές στην σταθεροποίηση του προβλήματος, με αποτέλεσμα να μην διαφέρει πολύ από την λύση χωρίς συνεκτικότητα. Παρουσιάζονται και οι δύο περιπτώσεις με ανάλογο σχολιασμό.

Μη συνεκτική ροή Στη ροή χωρίς τάσεις τριβής, οι στρόβιλοι χάνουν γρήγορα την μορφή τους, και πλησιάζοντας στο σύνορο εμφανίζουν διαταράξεις, μάλιστα για μεγαλύτερες χρονικές στιγμές, το πεδίο στροβιλότητας απειρίζεται λόγω των αριθμητικών σφαλμάτων και παρουσιάζονται φαινόμενα διάχυσης σε σημείο που ο υπολογισμός του δεν είναι πλέον δυνατός (overflow error).

Συνεκτική ροή Στη ροή με συνεκτικότητα, περιορίζεται η αριθμητική διάχυση, το σχήμα γίνεται ευσταθές και ακολουθεί μια περισσότερο ομαλή εξέλιξη στον χρόνο. Είναι γνωστό και από την θεωρία ότι η κινηματική συνεκτικότητα δρά ως όρος ευστάθειας για τα προβλήματα μη μόνιμων ροών.



Σχήμα 9: Στιγμιότυπα του μη μόνιμου μη συνεκτικού προβλήματος



Σχήμα 10: Στιγμιότυπα του μη μόνιμου συνεκτικού προβλήματος

Βιβλιογραφία

[1] Μπεργελές, Γ., (2012). Υπολογιστική Ρευστομηχανική, Εκδόσεις Συμεών: Αθήνα.

[2] Barba, Lorena A., and Forsyth, Gilbert F. (2018). CFD Python: the 12 steps to Navier-Stokes equations. Journal of Open Source Education, 1(9), 21, <https://doi.org/10.21105/jose.00021>.

[3] Owkes, Mark. A Guide to Writing Your First CFD Solver - Montana. Montana State University, https://www.montana.edu/codes/GuideToCFD_2020_02_28_v2.pdf.

Παράρτημα - Κώδικας Python

```
# -*- coding: utf-8 -*-
"""
Created on Tue Apr 19 23:50:33 2022

@author: Ioannis Skounakis - Aikaterini Margarita Ilektra Georgouli
"""

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as animation
import matplotlib.colors as mcol
import matplotlib.cm as cm
import pandas as pd
from collections import namedtuple
import time
from mpl_toolkits.axes_grid1 import make_axes_locatable #anchor colorbar
cbarcolors=["#0207fa", "#04dede", "#91f01d", "#ffff00", "#fc1605", "#820e04"]
cm1 = mcol.LinearSegmentedColormap.from_list("newmap", cbarcolors, N=256, gamma=1.5)
plt.rcParams['text.usetex'] = True
#####
##GENERAL FUNCTIONS##
def distance(x1,y1,x2,y2): #distance between 2D points, accepts 4 coords
#####
    a,b=np.array([x1,y1],dtype=object),np.array([x2,y2],dtype=object)
    return np.linalg.norm(a-b)
def showgrid(x,y): #plots the lines of the grid - debug
    global ax
    X,Y=np.meshgrid(x, y)
    plt.plot(X,Y, color='k',linewidth=".5")
    plt.plot(np.transpose(X),np.transpose(Y), color='k',linewidth=".5")
#####Initial  $\omega_0-\phi_0$ #####
def W0(x, y): #i
#####
    global x1,y1,x2,y2
    r1=distance(x,y,x1,y1)
    r2=distance(x,y,x2,y2)
    w1=(2-r1**2)*np.exp((1-r1**2)/2)
    w2=-(2-r2**2)*np.exp((1-r2**2)/2)
    return w1+w2
def psy0(x, y):
    global x1,y1,x2,y2
    r1=distance(x,y,x1,y1)
    r2=distance(x,y,x2,y2)
    psi1=np.exp((1-r1**2)/2)
    psi2=-np.exp((1-r2**2)/2)
    return psi1+psi2
##GRID-MESH FUNCTIONS##
#####
def num2coord(n, flag=False): #one number to ij or xy
```

```

global Nx, rangex, rangey, dx, dy
if type(n)==list:
    ii=rangex[0]+(n[0]-1)*dx
    jj=rangey[0]+(n[1]-1)*dy
    return [ii,jj]
else:
    ii=n%Nx
    if ii==0:
        ii=Nx
    jj=(n-ii)/Nx+1
    if flag:
        ii=rangex[0]+(ii-1)*dx
        jj=rangey[0]+(jj-1)*dy
    return [ii,jj]
#####
def deriv(ii,jj,dgrid): #derivative
    global dx,dy
    #derivative to x
    try:
        dfdx=(dgrid[ii+1][jj]-dgrid[ii-1][jj])/(2*dx)
    except KeyError:
        try:
            dfdx=(dgrid[ii+1][jj]-dgrid[ii][jj])/(dx) #forward
        except KeyError:
            dfdx=(dgrid[ii][jj]-dgrid[ii-1][jj])/(dx) #backward
    #derivative to y
    try:
        dfdy=(dgrid[ii][jj+1]-dgrid[ii][jj-1])/(2*dy)
    except KeyError:
        try:
            dfdy=(dgrid[ii][jj+1]-dgrid[ii][jj])/(dy) #forward
        except KeyError:
            dfdy=(dgrid[ii][jj]-dgrid[ii][jj-1])/(dy) #backward
    return dfdx,dfdy
#####
def ar2mesh(ar,A=False): #array to pandas mesh
    global Nx,Ny
    if A:
        return pd.DataFrame(ar,index=np.arange(1, Nx*Ny+1),columns=np.arange(1,Nx*Ny+1))
    return pd.DataFrame(ar,index=np.arange(Ny,0,-1),columns=np.arange(1,Nx+1))
#####
def buildb(W): #add w mesh values in b
    global Nx,Ny
    b=np.zeros(Nx*Ny)
    for j in range(2,Ny):
        for i in range(2,Nx):
            k = (j-1)*Nx+i-1
            b[k]=-W[i][j]
    return b
#####
def boundmat(b,w,A): #add boundary sums in b
    global Nx, Ny, ddx, ddy, dx, dy, x, y
    for ii in range(Nx*Ny):
        if not(-2*ddx-2*ddy == A[ii][ii]):
            X, Y = np.meshgrid(x, y)
            bX,bY=np.full((Ny-2,Nx-2),num2coord(ii+1,True)[0]),np.full((Ny-2,Nx-2),num2coord(ii+1,True)[1])
            distances=np.sqrt((bX-X[1:-1,1:-1])**2+(bY-Y[1:-1,1:-1])**2)
            b[ii]=-np.sum((1/(2*np.pi))*np.log(distances)*w[1:-1,1:-1]*dx*dy)
    return b
#####
def animateVORT(frames): #matplotlib animation

```

```

    global Wdt, psy, A, dt, cbarticks,x,y
    Wdt,psy=updateMESH(Wdt,psy,False,True)
#####
    ax.clear()
    CS=ax.contourf(x,y,Wdt,cbarticks,cmap=cm1,vmin=-4, vmax=4)
    ax.contour(x,y,Wdt, cbarticks, colors='black',linewidths=.5,vmin=-4, vmax=4)
    ax.axis("equal")
    cbar = fig.colorbar(CS, cax=cax, ticks=cbarticks)
    ax.set_title("Vorticity Field, t = "+str(round(frames*dt,2))+ "s",fontsize = 'small')
#####
def updateMESH(w,psy,plotting=True,viscosity=False):
    global na, dt, A, timestep,cm1,cbarticks
#####
    W=ar2mesh(w[:-1])
#####
    #Find u,v mesh, d $\omega$ /dx d $\omega$ /dy and eventually  $\omega(x,y,t+\Delta t)$ 
    U=np.gradient(psy,dy,dx)[0] #u=d $\psi$ /dy, v=-d $\psi$ /dx
    V=-np.gradient(psy,dy,dx)[1]
    Wx=np.gradient(w,dy,dx)[1]
    Wy=np.gradient(w,dy,dx)[0]
    Wxx=np.gradient(Wx,dy,dx)[1]
    Wyy=np.gradient(Wy,dy,dx)[0]
#####
    Wdt=w-dt*(U*Wx+V*Wy)+(na*(Wxx+Wyy) if viscosity else 0)
#####
    W=ar2mesh(Wdt[:-1])
#####
    #Boundary Conditions & right hand matrix (b)
    b=buildb(W) #b is the 1*NxNy matrix of results
    b=boundmat(b,w,A)
#####
    #Solving Linear System Ax=b where x = [ $\psi_1 \dots \psi_{NxNy}$ ]
    solution = np.linalg.solve(A,b)
    psy=solution.reshape(Ny,Nx)
#####

    if plotting:
        plt.figure(dpi=500)
        CS=plt.contourf(x,y,Wdt,cbarticks,cmap=cm1,vmin=-4, vmax=4)
        plt.contour(x,y,Wdt, cbarticks, colors='black',linewidths=.5,vmin=-4, vmax=4)
        plt.colorbar(CS,ticks=cbarticks)
        #showgrid(x,y)
        plt.axis('equal')
        plt.title("Vorticity Field, t = "+str(round(timestep*dt,2))+ "s",fontsize = 'small')
        plt.show()

    return Wdt,psy

##MESH INITIALIZATION##
na = 1.5e-4
x1,x2=0,0
y1,y2=1.5,-1.5 #Vortex Coordinates
rangex=[-5,5]
rangey=[-5,5]
Nx,Ny=100,100
dx,dy=((rangex[1]-rangex[0])/(Nx-1)),((rangey[1]-rangey[0])/(Ny-1))
ddx,ddy=1/(dx**2),1/(dy**2)
x = np.linspace(rangex[0],rangex[1], Nx)
y = np.linspace(rangey[0],rangey[1], Ny)
X, Y = np.meshgrid(x, y, sparse=True)

```

```
#####
##set up animation plotsn##
fig, ax = plt.subplots(dpi=400)
div = make_axes_locatable(ax)
cax = div.append_axes('right', '5%', '5%')
cbarticks=np.arange(-4,5,1)
dt=0.01 #time step
##W0##
w0 = W0(X,Y) #Initial Conditions for w
W=ar2mesh(w0[:::-1])
##SOLUTION MATRIX NxNy*NxNy dimensions##
#initialize matrices
A=np.identity(Nx*Ny) #A is the NxNy*NxNy matrix of coefficients
for j in range(2,Ny):
    for i in range(2,Nx):
        k = (j-1)*Nx+i-1 #numbering starts from 0
        A[k] [k-(Nx)], A[k] [k-1], A[k] [k], A[k] [k+1], A[k] [k+(Nx)]=ddy, ddx, -2*ddx-2*ddy, ddx, ddy
#####
#first iteration
b=buildb(W)
b=boundmat(b, w0, A)
solution = np.linalg.solve(A,b)
psy=solution.reshape(Ny,Nx)
#error calculations
psyerror = (psy-psy0(X,Y))/max(solution)
Wdt=w0
U=np.gradient(psy,dy,dx)[0] #u=dψ/dy, v=-dψ/dx
V=-np.gradient(psy,dy,dx)[1]
UV=np.sqrt(U**2+V**2)
#####
#Initial Mesh - Analytical Values - Plots Only
#####

fig1,ax1=plt.subplots(1,2,dpi=700,figsize=(9, 4.5), tight_layout=True)
fig1.suptitle(r"Analytical Vorticity \& Stream Function")
wcontour=ax1[0].contourf(x,y,Wdt,10,cmap=cm1,vmin=-4, vmax=4)
ax1[0].contour(x,y,Wdt, 10, colors='black',linewidths=.5,vmin=-4, vmax=4)
ax1[0].axis("equal")
cbar = fig1.colorbar(wcontour, ax=ax1[0])
ax1[0].set_title(r'Vorticity Field, $t = 0s$', fontsize = 'medium')
ax1[0].set_xlabel(r'$x$ axis')
ax1[0].set_ylabel(r'$y$ axis')

wcontour=ax1[1].contourf(x,y,psy0(X,Y),10,cmap=cm1)
ax1[1].contour(x,y,psy0(X,Y), 10, colors='black',linewidths=.5)
ax1[1].axis("equal")
cbar = fig1.colorbar(wcontour, ax=ax1[1])
ax1[1].set_title(r'Stream function, $t = 0s$', fontsize = 'medium')
ax1[1].set_xlabel(r'$x$ axis')
ax1[1].set_ylabel(r'$y$ axis')
#####
fig2,ax2=plt.subplots(dpi=700)
wcontour=ax2.contourf(x,y,psyerror,10,cmap=cm1)
ax2.contour(x,y,psyerror, 10, colors='black',linewidths=.5)
ax2.axis("equal")
cbar = fig2.colorbar(wcontour, ax=ax2)
ax2.set_title(r'Absolute Error', fontsize = 'medium')
ax2.set_xlabel(r'$x$ axis')
ax2.set_ylabel(r'$y$ axis')
#####
figc,axc=plt.subplots(1,2, dpi=700,figsize=(9, 4.5), tight_layout=True)
```

```

figc.suptitle(r"Analytical vs Numerical Stream Function")
wcontour=axc[0].contourf(x,y,psy0(X,Y),10,cmap=cm1)
axc[0].contour(x,y,psy0(X,Y), 10, colors='black',linewidths=.5)
axc[0].axis("equal")
cbar = fig1.colorbar(wcontour, ax=axc[0])
axc[0].set_title(r'Analytical',fontsize = 'medium')
axc[0].set_xlabel(r'$x$ axis')
axc[0].set_ylabel(r'$y$ axis')

wcontour=axc[1].contourf(x,y,psy,10,cmap=cm1)
axc[1].contour(x,y,psy, 10, colors='black',linewidths=.5)
axc[1].axis("equal")
cbar = fig1.colorbar(wcontour, ax=axc[1])
axc[1].set_title(r'Numerical',fontsize = 'medium')
axc[1].set_xlabel(r'$x$ axis')
axc[1].set_ylabel(r'$y$ axis')
#####
fig1,ax1=plt.subplots(1,3,dpi=700,figsize=(9, 3), tight_layout=True)
fig1.suptitle(r"Horizontal & Vertical Velocity Fields")
wcontour=ax1[0].contourf(x,y,U,10,cmap=cm1)
ax1[0].contour(x,y,U, 10, colors='black',linewidths=.5)
ax1[0].axis("equal")
cbar = fig1.colorbar(wcontour, ax=ax1[0])
ax1[0].set_title(r'Horizontal Velocity, u',fontsize = 'medium')
ax1[0].set_xlabel(r'$x$ axis')
ax1[0].set_ylabel(r'$y$ axis')

wcontour=ax1[1].contourf(x,y,V,10,cmap=cm1)
ax1[1].contour(x,y,V, 10, colors='black',linewidths=.5)
ax1[1].axis("equal")
cbar = fig1.colorbar(wcontour, ax=ax1[1])
ax1[1].set_title(r'Vertical Velocity, v',fontsize = 'medium')
ax1[1].set_xlabel(r'$x$ axis')
ax1[1].set_ylabel(r'$y$ axis')

wcontour=ax1[2].contourf(x,y,UV,10,cmap=cm1)
ax1[2].contour(x,y,UV, 10, colors='black',linewidths=.5)
ax1[2].axis("equal")
cbar = fig1.colorbar(wcontour, ax=ax1[2])
ax1[2].set_title(r'VeLOCITY Vector',fontsize = 'medium')
ax1[2].set_xlabel(r'$x$ axis')
ax1[2].set_ylabel(r'$y$ axis')
#####
'''
#plt.figure(dpi=700)
#ax2 = plt.axes(projection='3d')
#ax2.plot_surface(X, Y, Wdt, rstride=1, cstride=1,cmap=cm1, edgecolor='none')
#plotting directly
'''
'''
for timestep in range(100):
    Wdt,psy=updateMESH(Wdt,psy,True,False)
'''

#animating

#ani = animation.FuncAnimation(fig, animateVORT, int(round(10/dt,0)), blit=False, interval=1000*dt)
#ani.save("trial2.mp4")

```